

XLeRobot Digital Twin — AI / Algorithmic / Behavioral Model

Course: RBB2013 Digital Twin (May 2026)

Repository: <https://github.com/ManHazz/RBB2013-Digital-Twin>

Team members:

Name	Student ID
Muhammad Aiman bin Ahmad Hazimin	22011708
Hazieq Danial bin Roshihan Annuar	24006633
Muhammad Raziq bin Sufian	24006626
Ibrohim bin Ahmad Jaafar Sadzik	24006396
Ariq Danish bin Nor Razak	24006796

Rubric: AI or Algorithmic/Behavioral Model (5%)

"Demonstrate any AI or Behavioral model relevant to your digital twin. Show it is responsive to data ingested and updates digital twin state or any action."

1. Model 1 — LLM-based intent parser (`nl-command` service)

Purpose: Convert a free-form natural-language command into a structured target pose the rest of the pipeline can act on.

Model: Qwen2.5 3B parameters, served locally via Ollama (`http://host:11434/api/generate`).
Configurable via env var `OLLAMA_MODEL` .

1.1 Data ingested

- **User command string** — arrives via HTTP `POST /command` on port 8010.
- **System prompt** — describes valid arm actions and required output schema.
- **Fallback target pose** — used when the LLM emits only a "home" step.

1.2 Behavior — how it responds to input

1. Text arrives at `POST /command` on port 8010.
2. `nl-command` forwards a prompt+system to Ollama. The system prompt constrains the LLM to output only a JSON array of high-level action steps: `[{"action": "above|grab|lift|place|home", "wait": <seconds>}]` .
3. `nl-command` parses the LLM's raw completion, repairs malformed JSON (unquoted keys, `<think>` blocks) via `repair_json()` .
4. `nl-command` maps each action to an `(x, y, z)` target via `action_target()` :
 - `above` → target position with `y += 12 cm` (approach from above the ball).
 - `grab` → target position at the ball.
 - `lift` → target position with `y += 25 cm` (lift the ball).
 - `place` → target position (release).
5. Returns a `TargetPose(x, y, z)` — the digital twin's next commanded end-effector goal.

1.3 How it updates digital twin state

The returned `TargetPose` feeds into `motion-planner` → `dispatcher` → `sim-bridge`, where it becomes the new set of joint angles that update the arm in Omniverse **and** stream into TimescaleDB + Redis.

1.4 Failure modes exercised in tests

- Empty text → HTTP 422 (`tests/unit/test_nl_command.py::test_empty_text_returns_422`).
- Garbled LLM response → HTTP 422 (`tests/unit/test_nl_command.py::test_garbled_ollama_returns_422`).
- Ollama unreachable → HTTP 502.

In []: *# Live demo – send a natural-language command, receive a target pose*

```
import httpx, json

resp = httpx.post(
    "http://localhost:8010/command",
    json={"text": "pick up the ball"},
    timeout=60,
)
print("HTTP", resp.status_code)
print(json.dumps(resp.json(), indent=2))

# Expected output:
# HTTP 200
# {
#   "x": 40.0,
#   "y": 13.75,
#   "z": 0.0
# }
```

2. Model 2 — Inverse kinematics + collision checker (`motion_planner service, robot_ik.py`)

Purpose: Given a target end-effector pose, compute 6 joint angles that reach the pose without self-collision or obstacle collision.

Model type: Classical analytical/numerical IK with a spherical-approximation collision checker over arm segments and known obstacles. Public API in `services/motion_planner/robot_ik.py` :

- `forward_kinematics(joints)` -> `Point3D` — where the gripper ends up given joint angles.
- `gripper_tip(joints)` -> `Point3D` — end-effector position specifically.
- `solve(target)` -> `joints` — inverse: find joint angles that reach `target` .
- `reachable(target)` -> `bool` — target is within the arm's workspace.
- `accuracy_at(joints, target)` -> `float` — Euclidean distance between `FK(joints)` and `target`.
- `arm_points(joints)` -> `list` — spherical waypoints along the arm for collision checks.
- `collides(joints, obstacles)` -> `bool` — any arm sphere overlaps any obstacle sphere?

2.1 Data ingested

- `PlanRequest.target` — an (x, y, z) pose. Comes from `nl-command`'s LLM output, so ultimately from a user command.
- (Internal) hard-coded obstacle list from the scene.

2.2 Behavior — how it responds to input

1. `POST /plan` receives `{"target": {"x", "y", "z"}}` .
2. Calls `reachable(target)` . If `false` → returns `{joints: [0]*6, reachable: false, collision_free: true}` (HTTP 200, negative answer — not an error).
3. Calls `solve(target)` to get joint angles.
4. Calls `collides(joints, obstacles)` . If `true` → returns `{joints, reachable: true, collision_free: false}` .
5. Otherwise → returns `{joints, reachable: true, collision_free: true}` .

2.3 How it updates digital twin state

Returned joint angles are passed to `dispatcher` , which interpolates from current pose to the target over 30 frames at 30 fps, then ZMQ-pushes each frame to `sim-bridge`. `Sim-bridge` applies the angles → the digital twin arm moves in `Omniverse` → the sim tick publishes a new `SimState` → telemetry writes it to `TimescaleDB` + `Redis`. **State updated end-to-end.**

2.4 Failure modes exercised in tests (rubric requires pass AND fail cases)

- Reachable target → correct joints returned, forward-kinematics round-trip within $1e-3$ (`tests/unit/test_motion_planner.py::test_ik_roundtrip`).
- Unreachable target → `reachable=False` returned, no crash (`test_unreachable_target_rejected`).
- Colliding target → `collision_free=False` returned (`test_colliding_target_rejected`).
- Golden regression suite in CI: fixed target→(reachable, collision_free) pairs, IK output drift fails the build (`tests/regression/test_golden_ik.py`).

```
In [ ]: # Live demo – send the LLM's output pose to motion-planner, get 6 joint angles

import httpx, json

resp = httpx.post(
    "http://localhost:8020/plan",
    json={"target": {"x": 40.0, "y": 13.75, "z": 0.0}},
    timeout=15,
)
print("HTTP", resp.status_code)
print(json.dumps(resp.json(), indent=2))

# Expected output:
# HTTP 200
# {
#   "joints": [0.0, 1.10, -0.55, 2.58, 0.0, 0.0],
#   "reachable": true,
#   "collision_free": true
# }
```

3. Why both models are needed

Model	Role	Ingested data	Updates state
LLM (Qwen2.5 via Ollama)	Semantic — "what does the user want?"	User text	Produces TargetPose
IK + collision	Physical — "how do we get there safely?"	TargetPose	Produces joint angles that become the sim's new joint state

Neither is sufficient alone: an LLM can't do 6-DoF IK reliably; a solver can't understand English. Chained, they turn a sentence into a validated, collision-free trajectory that updates the digital twin.

4. Live end-to-end evidence

4.1 Omniverse — arm reaching the target after a natural-language command



User said "pick up the ball". The LLM parsed intent → the IK solved joints → the dispatcher streamed 30 interpolated frames → sim-bridge applied them → the gripper reached the red target ball.

4.2 Grafana — digital twin state updated in real time



The **Gripper position (cm)** panel shows the ee_x (Reach) trace climbing to 40 cm — the ball's location — after the command. The **Pipeline health** panel confirms ~600 state updates/min flowing from sim into TimescaleDB, driven entirely by the AI-model chain.

4.3 Terminal — the model chain end-to-end



Line 1: user's natural-language command. Line 2: motion-planner's joints result. Line 3: dispatcher accepted, streamed frames, and triggered actuation over MQTT.

5. Summary

- **AI/behavioral models used:** Qwen2.5 3B LLM + 6-DoF analytical IK + spherical-collision checker.
- **Responsive to data ingested:** LLM ingests user commands via HTTP; IK ingests target poses via HTTP.
- **Updates digital twin state:** joints stream to Omniverse (updates rendered arm) → sim publishes state → telemetry updates Redis (`state:latest`) and TimescaleDB (historical `robot_state`).
- **Tested at three tiers:** unit (pass + fail cases), integration (nl → planner contract), regression (golden IK cases in CI on every push).
- **Proven end-to-end:** live command → arm reaches ball → Grafana shows the trajectory + healthy pipeline.